Solution: Balanced Parentheses

Problem Statement

Given a string `s` containing `(`, `)`, `[`, `]`, `{`, and `}` characters. Determine if a given string of parentheses is balanced.

A string of parentheses is considered balanced if every opening parenthesis has a corresponding closing parenthesis in the correct order.

Example 1:

Input: String s = "{[()]}";

Expected Output: true

Explanation: The parentheses in this string are perfectly balanced. Every opening parenthesis '{', '[', '(' has a corresponding closing parenthesis '}', ']', ')' in the correct order.

Example 2:

Input: string s = "{[}]";

Expected Output: false

Explanation: The brackets are not balanced in this string. Although it contains the same number of opening and closing brackets for each type, they are not correctly ordered. The ']' closes '[' before '{' can be closed by '}', and similarly, '}' closes '{' before '[' can be closed by ']'.

Example 3:

Input: String s = "(]";

Expected Output: false

Explanation: The parentheses in this string are not balanced. Here, ')' does not have a matching opening parenthesis '(', and similarly, ']' does not have a matching opening bracket '['.

Solution

To solve this problem, we use a stack data structure. As we traverse the string, each time we encounter an opening parenthesis ('(', '{', or '['), we push it onto the stack. When we find a closing parenthesis (')', '}', or ']'), we check if it matches the type of the opening parenthesis at the top of the stack. If it matches, we pop the top element from the stack; if not, or if the stack is empty when we find a closing parenthesis, the string is not balanced, and we return `false`.

After processing the entire string, if the stack is empty, it means all parentheses were properly closed and nested, so we return `true`. Otherwise, we return `false`.

Here is a step-by-step algorithm:

1. Initialize an empty Stack.
2. Iterate over the string of parentheses.
   - If the current character is an opening parenthesis, push it onto the Stack.
   - If the current character is a closing parenthesis, check the top of the Stack.
     - If the Stack is empty, then the string is not balanced (there is a closing parenthesis without a matching opening parenthesis), so return false.
     - If the top of the Stack is the matching opening parenthesis, pop it off the Stack.
     - If the top of the Stack is not the matching opening parenthesis, then the string is not balanced, so return false.
3. After checking all parentheses, if the Stack is empty, then the string is balanced, so return true. If the Stack is not empty, then there are unmatched opening parentheses, so the string is not balanced, return false.

Algorithm Walkthrough

Let's consider the input "{[()]}", and observe how algorithm works.

Initialization:

- Start with an empty stack.

Iteration 1: Character = '{'

- Stack before operation: []
- Since '{' is an opening bracket, push it onto the stack.

Iteration 2: Character = '['

- Stack before operation: ['{']
- Since '[' is an opening bracket, push it onto the stack.

Iteration 3: Character = '('

- Stack before operation: ['{', '[']
- Since '(' is an opening bracket, push it onto the stack.

Iteration 4: Character = ')'

- Stack before operation: ['{', '[', '(']
- ')' is a closing bracket. The top of the stack is '(', which is the corresponding opening bracket for ')'. So, pop '(' from the stack.

Iteration 5: Character = ']'

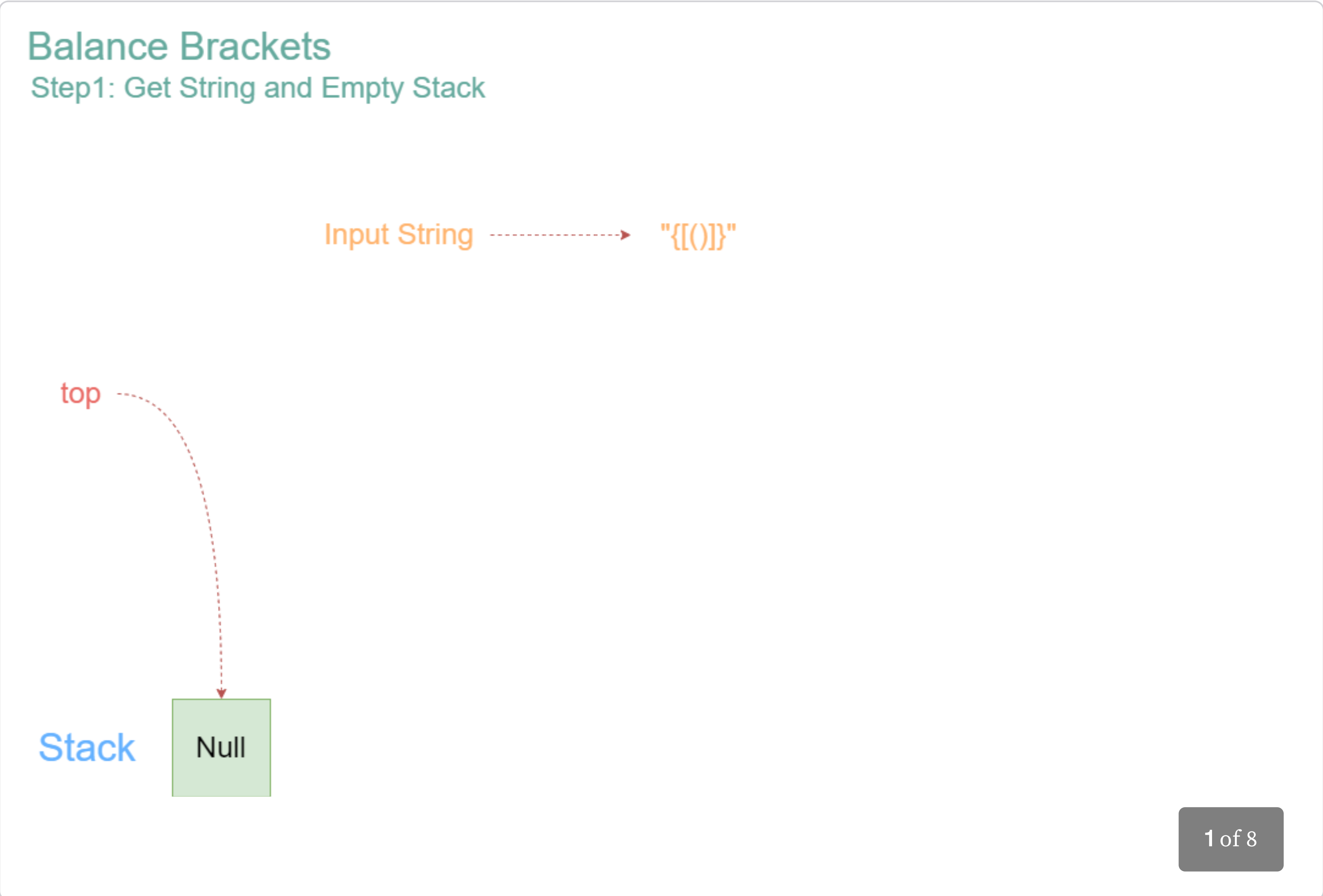
- Stack before operation: ['{', '[']
- ']' is a closing bracket. The top of the stack is '[', which is the corresponding opening bracket for ']'. So, pop '[' from the stack.
-

Iteration 6: Character = '}'

- Stack before operation: ['{']
- '}' is a closing bracket. The top of the stack is '{', which is the corresponding opening bracket for '}'. So, pop '{' from the stack.

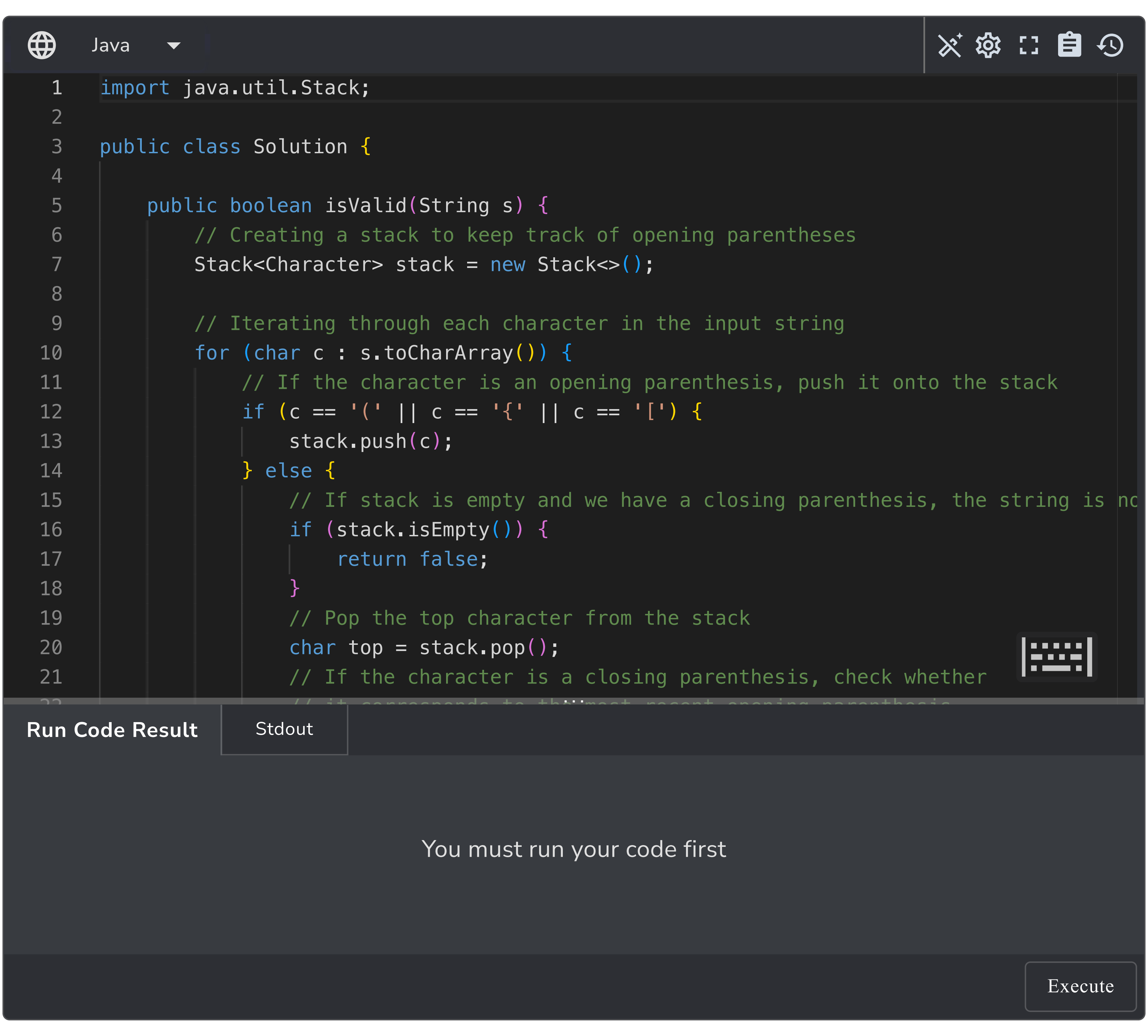
Final Check:

- After processing all characters, check the stack.
- The stack is empty, indicating that all opening brackets were properly matched and closed.
- Therefore, the input string "{[()]}" is valid with properly balanced parentheses.



Code

Here is how you can implement this algorithm:



```java
import java.util.Stack;

public class Solution {

    public boolean isValid(String s) {
        // Creating a stack to keep track of opening parentheses
        Stack<Character> stack = new Stack<>();

        // Iterating through each character in the input string
        for (char c : s.toCharArray()) {
            // If the character is an opening parenthesis, push it onto the stack
            if (c == '(' || c == '{' || c == '[') {
                stack.push(c);
            } else {
                // If stack is empty and we have a closing parenthesis, the string is n
                if (stack.isEmpty()) {
                    return false;
                }
                // Pop the top character from the stack
                char top = stack.pop();
                // If the character is a closing parenthesis, check whether
```

In this code, the `isValid` function checks if a given string of parentheses `s` is balanced by using a Stack. It pushes any opening parentheses onto the Stack and pops off the Stack whenever a closing parenthesis is encountered. If the Stack is empty at the end, then the string is balanced.

Time and Space Complexity Analysis

The time complexity of this algorithm is O(n), where n is the length of the string. This is because we're processing each character in the string exactly once.

The space complexity is also O(n) in the worst-case scenario when all the characters in the string are opening parentheses, so we push each character onto the Stack. In the average case, however, the space complexity would be less than O(n).

← Previous
Problem 1: Balanced Parentheses

Next →
Problem 2: Reverse A String

☐ Mark as Completed